

PivotMap

What survives when a requirement changes after the code is written.

A guide to the command, the agent skill, and the MCP server · BTW Buildathon 2026 · Track 2

Someone changes the rules after the work is done. Legal drops a vendor. Compliance bans a library. An architecture rule now forbids one part of the system from touching another. The code is already written, and somebody has to answer three questions before anyone touches anything: **what actually breaks this rule, what is built on top of that, and what can be left alone?**

Today that is manual archaeology. `git log` shows what changed but not why anyone chose it. Searching finds the banned name but not the three modules quietly built on top of it. So the work is either over-done — treating the whole area as suspect — or under-done, and something breaks in production.

PivotMap answers those three questions from a map of the codebase, and — this is the part that matters — **it tells you how much of its own answer it is actually sure about.**

Three doors, one engine

The same analysis is reachable three ways, because the people who need it work in different places.

FOR DEVELOPERS

The command

A terminal command. Full detail: every file, every piece of evidence, the exact line that justifies each verdict.

FOR CODING AGENTS

The skill

Teaches an AI coding assistant when to run the analysis and — more importantly — how much to trust each part of the answer.

FOR EVERYONE ELSE

The MCP server

Makes the analysis callable from a chat window. A product manager asks in plain English and gets a decision brief, not file paths.

The rule it answers

Every constraint PivotMap understands has the same shape:

`X may no longer reach Y`

That one sentence covers more than it looks like it does:

The constraint	X	Y
"We're dropping Stripe"	everything	the Stripe library

"This library may not touch customer data"	everything	the library
"The API layer may not reach storage directly"	the API layer	storage
"Nothing outside payments may use the payments internals"	everything else	payments internals

What it does not answer, and this is a real boundary rather than a missing feature: constraints about *behaviour* rather than dependency — "no background workers", "no personal data in logs" — and migrations, where the question is what to build rather than what breaks. Those need different analysis, not a different setting.

What the answer looks like

Every file in the codebase lands in one of five buckets:

Verdict	Plain meaning	What to do
Directly affected INVALIDATED	This file itself breaks the rule.	Must change. Start here.
Needs validation AT-RISK	Breaks no rule itself, but stands on something that does.	Re-check <i>after</i> the first group is fixed, not before.
No impact found SAFE	Read cleanly, and no connection found.	Leave alone.
Not assessed UNVERIFIED	The analyzer could not read this file at all.	Not cleared. Needs a human.
Not assessed UNREACHED	Written in a language the analyzer only inventories.	Unknown, not safe.

The fourth row is the important one. Before this release, a file the analyzer had failed to read was reported as *safe* — "no connection found, leave alone". Five files in this project's own codebase were being reported that way while the parser was erroring on them. That is a gap in the analysis being presented as a finding about the code.

How sure is it? — the part most tools skip

A map of a codebase is built by reading the source. Some of what it finds is a fact it read directly. Some is an educated guess. **Until this release, both were shown the same way.**

Now every piece of evidence carries a grade:

Grade	Means	What you should do
CONFIRMED	Read straight from the source. An import statement, or a call followed to its definition.	Act on it.
HEURISTIC	Inferred — matched by name or by package. Often right. Never proof.	Open the cited line and check before acting.
UNVERIFIED	The file could not be read properly.	Treat as unexamined.

Why this is not academic

Run against Kubernetes — 17,838 files of Go — the analysis produced 52 confirmed connections and **71 guesses**. More of the answer was inferred than proved. One of those guesses was demonstrably wrong:

```
graph.go      func (n *node) setOwners(owners []OwnerReference)
                ^^^^^^ a variable name

prunexml.go   type owners struct
                ^^^^^^ an unrelated type, in a different program
```

The analyzer matched a *variable* called `owners` to a *type* called `owners` in a completely unrelated file — one the first file does not reference at all. Before this release that false connection was displayed identically to a real import statement. Now it is tagged as a guess, with a line telling the reader to check it.

This is why "no connection found" is never reported as proof that no connection exists. Some code is wired together at runtime in ways no amount of reading can see. The report says *no path was found*, which is a different sentence.

Door 1 — the command

```
# build it once (needs Go)
mise run build

# "we're dropping this library"
./entire-graph pivot --dependency github.com/stripe/stripe-go

# "the CLI layer may not reach the graph layer"
./entire-graph pivot --from internal/cli --dependency internal/sem
```

Option	What it is
<code>--dependency</code>	Y — what may no longer be reached. Required.

<code>--from</code>	X — who the rule applies to. Leave out for a rule that applies everywhere.
<code>--depth 1..5</code>	How far to follow the ripple. 1 gives a shortlist.
<code>--exclude-tests</code>	Ignore test files.
<code>--format plan</code>	Output a work order for an AI agent instead of a report.

Scoping matters more than it sounds. In this project, "nothing may reach the graph layer" returns 139 affected files. "The *CLI* layer may not reach the graph layer" returns 23. The other 116 are real, legitimate dependencies — they are simply not what the rule was about, and listing them buries the answer.

Door 2 — the agent skill

A skill is a short instruction file an AI coding assistant loads automatically when it becomes relevant. Ours fires on phrases like "we can't use *X* anymore" or "legal dropped this vendor".

Its value is not the commands — those are easy. It is the **discipline**:

- Run the analysis first, rather than searching around and guessing.
- Before acting on a guess, open the cited line and confirm it.
- A file the analyzer could not read is *unread*, not *cleared*.
- When reporting the work done, re-run the analysis and **show the change alongside the number** — because "zero problems remaining" is also achieved by deleting the code.

It is named `what-survives`, to pair with an existing Entire skill called `what-happened`. That one looks backward — *why does this code look the way it does?* This one looks forward — *what still stands, now the rules have changed?*

Door 3 — the MCP server, for people who don't use a terminal

MCP is a standard way for an AI assistant to call an external tool. Registering this server makes the analysis available inside a chat window — no terminal, no commands.

Why that matters: constraints usually arrive from people who cannot cost them. Legal drops a vendor; compliance bans a library; a requirement changes. Each of those people currently has to find a developer to translate the constraint into an answer.

Three of the seven tools are written for that audience, and they follow strict rules:

- **Plain language only.** Never "SAFE" — a green word reads as *proof* to someone without the instinct to distrust it. It says "no impact found in analyzed code", and says what was not analyzed in the same breath.
- **No effort estimates.** Ever. It reports *shape* — concentrated in one area, or spread across nine — and a decision class such as *discovery required*. A number of hours derived from a dependency map is false precision.

- **It refuses to guess.** If "Acme Analytics" could mean three different packages, it lists them and stops. A confident, specific, wrong answer is the failure this tool exists to prevent.

What a product manager actually gets back

DECISION BRIEF

8 places across 5 areas use this directly, with 101 further places needing re-checking afterwards.

SCOPE CLASS: Discovery required

Driven by: 8 direct uses found, but 92 files were not assessed (31% of the analyzed set), so the picture is incomplete.
This is a shape, not a schedule.

EVIDENCE AND CONFIDENCE

Evidence strength	Mostly inferred. Only 23 connections were read directly from source; 86 were matched by name.
Analysis coverage	Partial. 92 files could not be assessed.
Product interpretation	Weakest of the three – confirm with an owner.

Three separate confidence statements, because a single number would hide the difference between "*the evidence is weak*", "*we couldn't see everything*", and "*we're guessing which product area this maps to*". Those are different problems with different fixes.

Checking the work afterwards

When an AI agent does the remediation, something has to decide whether it actually worked. That check is **ordinary code, not another AI** — an AI grading an AI just moves the trust problem somewhere else.

It returns one of four answers:

Result	Means
PASS	Every check ran and passed.
FAIL	A check ran and did not pass.
PARTIAL	What could be checked, passed — but some things cannot be checked mechanically. <i>This is the normal good outcome.</i>
CANNOT VERIFY	A check could not run at all. Never treated as a pass.

The reason it exists: "zero problems remaining" is satisfied just as well by **deleting the code** as by fixing it. Tested for real — deleting the offending file makes the original check report success, and the new checks catch it on two independent grounds.

What it cannot do

Stated plainly, because a tool that hides its limits is the thing this project exists to argue against.

- **Code wired together at runtime is invisible.** Some connections are decided while the program runs, and leave nothing to read. A genuinely affected file can come back as "no impact found".
- **It does not interpret sentences.** It takes a package or module name. Turning "we can't send customer data through Library X" into that name is a human or assistant step, deliberately kept outside the tool — which is what lets it run entirely offline, with no data leaving the machine.
- **Behavioural rules are out of scope.** "No background workers" is not a question about what depends on what.
- **It cannot tell you how long the work will take.** By design.

PivotMap is built on Entire Graph, an offline code-analysis tool. No network access, no AI model in the analysis path, no data leaves the machine. Every verdict is a rule applied to evidence that is printed alongside it, so any claim in the report can be checked rather than trusted.